

Railway Depot Shunting

Problem 1

Note the dataset `unit_arrival_departure.info.jl` must be downloaded from DTU Learn, it is not available on the book website.

Sets

- $t \in Types = \{1, 2\}$: unit types
- $u \in Units = \{1, 2, \dots, U\}$: train units
- $k \in Tracks = \{1, 2, \dots, K\}$: tracks

Parameters

- $Type_u$: type of unit u
- $Arrival_u$: arrival time of unit u
- $Depart_u$: departure time of unit u
- $Length_u$: length unit u (depends on type)
- $TrackLength_k$: length of track k
- $Conflict_{u_1, u_2}$ Incidence Matrix where $Conflict_{u_1, u_2} = 1$ if units u_1 and u_2 are in conflict and 0 otherwise. Note that these can be pre-calculated based on arrival- and departure times.
- $InDepot_{u_1, u_2}$ if unit u_1 is in depot when unit u_2 arrives. Can also be pre-calculated based on arrival and departure times.

Decision variables

- $x_{u,k} \in \{0, 1\}$: if unit u is assigned track k (=1) or not (=0)

Model

Objective:

- Maximize the number of units that can be parked:

$$\sum_{u=1}^U \sum_{k=1}^K x_{uk}$$

Constraints:

- Each unit can be parked on at most one track

$$\sum_{k=1}^K x_{uk} \leq 1 \quad \forall u = 1, 2, \dots, U$$

- Collective length of simultaneously parked units on a track cannot exceed track length:

$$\sum_{j=1}^U InDepot[j, u] \cdot Length_j \cdot x_{jk} \leq TrackLength_k \quad \forall u = 1, \dots, U, \forall k = 1, 2, \dots, K$$

- Prevent track conflicts:

$$x_{u_1 k} + x_{u_2 k} \leq 1 \quad \forall u_1, u_2 \mid Conflict[u_1, u_2] = 1$$

Note: For the track length constraint we need to ensure that the length of the track is not exceeded at any point in time. However, instead of needing to have time-specific variables and check every minute we only check every time a new train is added, i.e. every time a train arrives. The constraint can then be understood as: "For each track, k , and for each train unit, u : At the time where u arrives to the depot, check that the collective length of trains that are currently parked on track k , does not exceed the track length."

Note: To see how the $Conflict_{u_1, u_2}$ values can be pre-calculated see the code below.

The full model in Julia/JuMP is given below

```
*****
# Railway depot shunting, "Mathematical Programming Modelling" (42112)
# Basic model
using JuMP
using HiGHS
#using Gurobi
```

```

*****

include("unit_arrival_departure_info.jl")
#Includes Type, arrival, adn depart for each of the 70 units.

Length = [42, 84] #Length of unit type 1 and 2
trackLength = [500, 800, 700, 400, 650, 650, 750, 500]

K=length(trackLength) #Number of tracks
U=length(type) #Number of train units

RS=Model(HiGHS.Optimizer)

*****
#Pre-calculations:

InDepot=zeros(U,U)
# identify whether two units are present in depot at same time
for u=1:U
    for j=1:U
        if arrival[j] <= arrival[u] && arrival[u] < depart[j]
            InDepot[u,j]=1
        end
    end
end

Conflict=zeros(U,U)
# calculate unit conflicts (cannot share a track)
for u=1:U
    for j=1:U
        if u < j
            if arrival[u]<=arrival[j]
                if depart[u] < depart[j] && depart[u] > arrival[j]
                    Conflict[u,j]=1
                end
            end
            if arrival[j]<=arrival[u]
                if depart[j] < depart[u] && depart[j] > arrival[u]
                    Conflict[u,j]=1
                end
            end
        end
    end
end
end

```

```

*****
# Model:
# assignment variable unit to track:
@variable(RS, x[u=1:U, k=1:K], Bin)

#maximize the number of units that can park
@objective(RS, Max, sum(x[u,k] for u=1:U for k=1:K))

# each unit assigned to at most one track:
@constraint(RS, [u=1:U], sum(x[u,k] for k=1:K)<=1)

# track length restriction (checked at each arrival):
@constraint(RS, [u=1:U, k=1:K], sum(InDepot[u,j]*Length[type[j]]*x[j,k] for j=1:U)
<= trackLength[k])

# unit conflict constraints
@constraint(RS, [u=1:U-1, j=u+1:U, k=1:K; Conflict[u,j]==1], x[u,k]+x[j,k]<=1)

optimize!(RS)

if termination_status( RS) == MOI.OPTIMAL
    println("Number of Parked Units: $(round((objective_value(RS)),digits=1))")
    for k=1:K
        print("Track $k:")
        unitsT = []
        for u=1:U
            if value(x[u,k])>1-1E-6
                push!(unitsT, (arrival[u],depart[u],type[u]))
            end
        end
        sort!(unitsT, by = first)
        println(unitsT)
    end
end
end

```

Problem 2

New Variables

- y_{kt} track k is assigned units of type t

New Constraints

- Each track is assigned at most 1 type of unit:

$$\sum_{t=1}^T y_{kt} \leq 1 \quad \forall k \in K$$

- Only units of the assigned type can be parked on the track:

$$x_{uk} \leq y_{kt} \quad \forall u \in U, k \in K, t \in T \mid Type_u = t$$

The full model in Julia/JuMP is given below

```

#####
# Railway depot shunting 2, "Mathematical Programming Modelling" (42112)
# Basic model
using JuMP
using HiGHS
#using Gurobi
#####

include("unit_arrival_departure_info.jl")
#Includes Type, arrival, adn depart for each of the 70 units.

Length = [42, 84] #Length of unit type 1 and 2
trackLength = [500, 800, 700, 400, 650, 650, 750, 500]

K=length(trackLength) #Number of tracks
U=length(type) #Number of train units
T=length(Length) #Number of types of units

RS=Model(HiGHS.Optimizer)

#####
#Pre-calculations:

InDepot=zeros(U,U)
# identify whether two units are present in depot at same time
for u=1:U
    for j=1:U
        if arrival[j] <= arrival[u] && arrival[u] < depart[j]
            InDepot[u,j]=1
        end
    end
end
end

```

```

Conflict=zeros(U,U)
# calculate unit conflicts (cannot share a track)
for u=1:U
    for j=1:U
        if u < j
            if arrival[u]<=arrival[j]
                if depart[u] < depart[j] && depart[u] > arrival[j]
                    Conflict[u,j]=1
                end
            end
            if arrival[j]<=arrival[u]
                if depart[j] < depart[u] && depart[j] > arrival[u]
                    Conflict[u,j]=1
                end
            end
        end
    end
end

*****
# Model:
# assignment variable unit to track:
@variable(RS, x[u=1:U, k=1:K], Bin)

# track type choice (track is for units of type t)
@variable(RS, y[k=1:K, t=1:T], Bin)

# maximize the number of units that can park
@objective(RS, Max, sum(x[u,k] for u=1:U for k=1:K))

# each unit assigned to at most one track:
@constraint(RS, [u=1:U], sum(x[u,k] for k=1:K)<=1)

# track length restriction (checked at each arrival)
@constraint(RS, [u=1:U, k=1:K], sum(InDepot[u,j]*Length[type[j]]*x[j,k] for j=1:U)
    <= trackLength[k])

# unit conflict constraints
@constraint(RS, [u=1:U-1, j=u+1:U, k=1:K; Conflict[u,j]==1], x[u,k]+x[j,k]<=1)

# dedicated track constraints:
@constraint(RS, [k=1:K], sum(y[k,t] for t=1:T) <=1)
@constraint(RS, [k=1:K, t=1:T, u=1:U; type[u]==t], x[u,k]<=y[k,t])

optimize!(RS)

```

```

if termination_status(RS) == MOI.OPTIMAL
    println("Number of Parked Units: $(round((objective_value(RS)),digits=1))")
    for k=1:K
        print("Track $k:")
        unitsT = []
        for u=1:U
            if value(x[u,k])>1-1E-6
                push!(unitsT, (arrival[u],depart[u],type[u]))
            end
        end
        sort!(unitsT, by = first)
        println(unitsT)
    end
end
end

```